ALLIANCE UNIVERSITY • SCHOOL OF COMPUTING • DEPARTMENT OF COMPUTER APPLICATIONS

# Assignment 4: Relational Database vs. NoSQL Database

**Comparative Analysis, Enterprise Rationale & Real-World Implementations**

**Student Name:** Sanjeev Adarsh S
**Program / Semester:** BCA • Semester III
**Course:** Database Management Systems (DBMS Lab)
**Deliverable:** Individual Assignment 4 (PDF)

## 1. Executive Summary

The database landscape has evolved significantly over the past two decades. For over thirty years, **Relational Database Management Systems (RDBMS)** based on Edgar F. Codd's relational model dominated enterprise applications through rigid schemas, structured query language (SQL), and strict ACID guarantees. However, the exponential growth of web-scale applications, unformatted big data, microservices, and global distributed computing spurred the emergence of **NoSQL (Not Only SQL)** databases.

This comparative analysis evaluates relational and NoSQL paradigms across four critical engineering dimensions: **scalability**, **consistency**, **query performance**, and **application use cases**. Furthermore, it articulates why modern organizations adopt specific data technologies and explores how our **Smart Library Management System (SLMS)** leverages a *polyglot persistence* architecture to balance relational integrity with distributed performance.

## 2. Architectural Comparison: RDBMS vs. NoSQL

| Dimension | Relational Databases (RDBMS) | NoSQL Databases |
|---|---|---|
| **Data Model & Schema** | Rigid, tabular structure composed of relations (tables), rows (tuples), and strictly typed columns. Enforces predefined schemas with primary and foreign keys. | Flexible, schema-less or schema-on-read formats. Categorized into Document (JSON/BSON), Key-Value, Wide-Column, and Graph models. |
| **Scalability Architecture** | **Vertical Scalability (Scale-Up):** Requires adding more CPU cores, RAM, and SSD capacity to a single master server. Horizontal sharding is complex and compromises join operations. | **Horizontal Scalability (Scale-Out):** Designed natively to partition and distribute data across clusters of commodity hardware nodes via automatic hashing and replication. |
| **Consistency Model** | **ACID Guarantees:** Strong immediate consistency. Guarantees Atomicity, Consistency, Isolation, and Durability across complex multi-table transactions. | **BASE & Eventual Consistency:** Adheres to Basically Available, Soft state, Eventual consistency. Sacrifices immediate consistency for partition tolerance (CAP Theorem). |
| **Query Language & Joins** | Standardized SQL. Excels at complex queries, multi-table joins, views, window functions, and aggregation pipelines. | No universal standard. Uses proprietary APIs or query languages (e.g., MQL for MongoDB, CQL for Cassandra). Joins are typically unsupported or pushed to application logic. |
| **Storage & Normalization** | Highly normalized (3NF/BCNF) to eliminate data redundancy and preserve referential integrity. | Deliberately denormalized. Embeds related entities within single documents or rows to enable single-lookup read efficiency. |
| **Prominent Examples** | PostgreSQL, MySQL, Oracle Database, Microsoft SQL Server, SQLite. | MongoDB, Apache Cassandra, Redis, Amazon DynamoDB, Neo4j, Couchbase. |

## 3. In-Depth Comparative Dimensions

### 3.1 Scalability: Vertical Scale-Up vs. Horizontal Scale-Out

Relational databases were engineered during an era when applications resided on single centralized mainframes or high-performance servers. Maintaining cross-table relational integrity, foreign key validation, and two-phase commit transactions becomes computationally prohibitive when data is scattered across independent network partitions. As a result, RDBMS systems traditionally rely on **vertical scaling**—upgrading CPU clocks, adding multi-terabyte RAM pools, and attaching high-speed SAN/NVMe arrays. However, vertical scaling faces a hard physical ceiling and exponential financial costs.

In contrast, NoSQL architectures were built from the ground up to solve web-scale storage challenges. Systems such as Apache Cassandra and Amazon DynamoDB use *consistent hashing rings* to partition keys across dozens or thousands of commodity server nodes. When data storage or query throughput outgrows existing cluster capacity, administrators simply boot additional worker nodes. The distributed cluster automatically rebalances partitions with zero scheduled downtime.

### 3.2 Data Consistency: ACID vs. BASE (CAP Theorem Trade-Offs)

The core trade-off between RDBMS and NoSQL is governed by Eric Brewer's **CAP Theorem**, which states that a distributed data store can simultaneously guarantee at most two out of three properties: **Consistency (C)**, **Availability (A)**, and **Partition Tolerance (P)**.

- **RDBMS (ACID Focus • CA Systems):** Relational databases prioritize consistency and isolation. If a student attempts to borrow the last remaining physical copy of *"Database System Concepts"* while another student attempts the exact same action, row-level locks guarantee that only one transaction commits. The inventory count decrements atomically, preventing inventory overdraw. In network partition events, relational nodes refuse writes to avoid inconsistent split-brain states.
- **NoSQL (BASE Focus • AP / CP Systems):** Most NoSQL engines opt for Availability and Partition Tolerance (AP). Under the **BASE** paradigm:
  - *Basically Available:* The system guarantees response availability across nodes, even in the presence of network degradations.
  - *Soft State:* Data values may change over time without immediate user interaction due to asynchronous background node synchronization.
  - *Eventual Consistency:* Replicas are not updated synchronously; however, in the absence of new mutations, all replicas eventually converge to the same value.

### 3.3 Query Performance & Latency

Query performance varies drastically based on the access pattern:

- **Relational Strengths:** RDBMS engines utilize sophisticated query optimizers, cost-based execution plans, and multi-column B-Tree or GiST indexes. They excel at multi-dimensional analytical queries, aggregate computations, and complex relational joins across 5 or more tables. However, as table sizes grow into billions of rows, join execution times degrade significantly due to heavy disk I/O and memory cache misses.
- **NoSQL Strengths:** Key-Value stores (like Redis) deliver sub-millisecond, \(O(1)\) read/write latency directly from in-memory hash tables. Document databases (like MongoDB) retrieve all parent-child details in a single sequential disk seek because child records are embedded inside the parent document (e.g., embedding author objects inside a book document). However, if an application requires cross-collection reporting, NoSQL queries become cumbersome and require multi-stage MapReduce or aggregation pipelines.

### 3.4 Workload Characteristics & Target Use Cases

- **Best Suited for RDBMS:** Systems requiring mathematical exactness, non-negotiable auditability, and structured relationships—such as banking ledgers, core university student enrollment, library circulation, ERP inventory, and hospital patient records.
- **Best Suited for NoSQL:** Systems handling high-velocity streaming data, dynamic schemas, and massive concurrency—such as IoT sensor feeds, real-time gaming leaderboards, social media activity streams, user clickstream logs, and shopping cart session state.

## 4. Why Organizations Choose Different Database Technologies

Modern enterprises do not treat database selection as a binary, dogmatic choice. Instead, technology leaders evaluate five primary architectural criteria:

1. **Transaction Integrity & Financial Rigor:** If data corruption or partial transaction commits lead to legal, financial, or severe academic penalties, relational ACID compliance is mandatory.
2. **Data Velocity & Volume:** If the workload generates terabytes of append-only events per hour (e.g., web analytics, smart sensor telemetry), a horizontally partitionable NoSQL engine is essential.
3. **Schema Volatility:** If the data structure evolves rapidly or incorporates heterogeneous semi-structured JSON payloads from external third-party APIs, schema-less document stores accelerate developer iteration.
4. **Query Diversity & Ad-Hoc Reporting:** If business executives require ad-hoc queries, cross-departmental dashboards, and slicing-and-dicing across disparate dimensions, relational SQL remains unmatched.
5. **Latency SLAs:** Applications requiring microsecond response times for session authorization or real-time caching mandate in-memory NoSQL stores.

## 5. Real-World Case Studies

### Case Study 1: Netflix • Global Streaming Telemetry & Subscription Billing

Netflix exemplifies large-scale polyglot persistence across hundreds of millions of global subscribers:

- **NoSQL Component (Apache Cassandra):** Netflix captures viewing history, video playback pause-points, and real-time user recommendations across global AWS regions using Apache Cassandra. The system writes petabytes of telemetry data daily without locking, benefiting from Cassandra's masterless, peer-to-peer linear scalability and high write throughput. Temporary eventual consistency is acceptable—if a user's viewing history takes 500 milliseconds to sync to a European server node, user experience is unaffected.
- **Relational Component (Amazon Aurora RDBMS):** For customer credit card billing, recurring subscription charges, legal licensing contracts, and partner royalties, Netflix relies on relational RDBMS engines. ACID guarantees ensure that payments are charged exactly once and financial journals remain mathematically balanced.

### Case Study 2: Amazon E-Commerce • Shopping Cart State vs. Financial Settlement

Amazon operates the world's largest retail platform by decoupling customer browsing from financial fulfillment:

- **NoSQL Component (Amazon DynamoDB):** Amazon's product catalog search, customer shopping carts, and session authorization tokens run on DynamoDB. During peak holiday events (e.g., Prime Day), millions of concurrent shoppers add items to carts simultaneously. DynamoDB guarantees predictable single-digit millisecond latency regardless of traffic surges.
- **Relational Component (Amazon RDS / PostgreSQL):** Once a customer clicks *"Place Order"*, the checkout workflow commits the purchase into a relational transaction engine. Inventory counts are locked and decremented, payment tokens are cleared, and tax invoices are generated under strict serializable isolation.

### Case Study 3: Smart Library Management System (SLMS) • Polyglot Architecture

Within our proposed **Smart Library Management System (SLMS)** at Alliance University, database requirements naturally bifurcate into two distinct operational profiles:

- **Core Relational Store (PostgreSQL / MySQL):** The backbone of SLMS is an ACID-compliant relational database. It manages catalog metadata (books, publishers, authors), student and faculty profiles, circulation checkouts, copy reservation queues, and fine penalty settlements. Relational integrity constraints ensure that no member borrows beyond their entitlement quota and no book copy is issued twice concurrently.
- **NoSQL In-Memory Cache (Redis):** As university students perform frequent keyword and title searches across the library OPAC (Online Public Access Catalog) before semester exams, querying the relational database for every keystroke creates severe CPU and disk bottleneck. SLMS deploys Redis as a secondary distributed cache. Frequently searched book summaries, category rankings, and active member session tokens are cached in-memory with a Time-To-Live (TTL) of 15 minutes, cutting database read load by over 80%.

## 6. Comparative Decision Matrix & Conclusion

| Selection Criterion | Relational Database Priority | NoSQL Database Priority |
|---|---|---|
| **Data Schema** | Stable, structured, highly relational entities. | Rapidly evolving, semi-structured, polymorphic data. |
| **Concurrency Model** | Pessimistic / Optimistic locking, strong isolation. | Lock-free, distributed conflict resolution (e.g., LWW). |
| **Transaction Boundary** | Multi-table atomic transactions. | Single-document / single-key atomic transactions. |
| **Infrastructure Cost** | High cost for high-end enterprise hardware. | Cost-effective scale-out on commodity cloud VMs. |

> ***Key Finding:*** *Neither relational nor NoSQL databases represent a universal solution. While RDBMS remains the gold standard for transactional consistency, referential integrity, and institutional auditability, NoSQL architectures provide the elasticity, horizontal partitioning, and low latency demanded by modern real-time platforms. Modern software architecture increasingly converges on hybrid polyglot persistence.*